

Politechnika Łódzka

Wydział Fizyki Technicznej, Informatyki i Matematyki Stosowanej

Instytut Informatyki

StegoCloud

**System mikrosерwisowy do ukrywania i odczytywania
szyfrowanych znaków wodnych w obrazach PNG**

Dokumentacja techniczna i projektowa

Przedmiot: Zaawansowane Zagadnienia Programowania w Javie

Członkowie zespołu projektowego:

Bartosz Kołaciński index: 251554

Mateusz Kosowski index: 251558

Nikodem Nowak index: 251598

Wiktor Pankanin index: 251606

Jakub Rosiak index: 251620

Jakub Rusek index: 247774

Łódź, Czerwiec 2026

Spis treści

1	Wprowadzenie	2
1.1	Cel projektu	2
1.2	Zakres projektu	2
1.3	Grupa docelowa (Do kogo projekt jest skierowany)	2
2	Opis systemu	3
2.1	Przeznaczenie	3
2.2	Główne funkcje	3
2.3	Architektura systemu	3
2.4	Integracje z zewnętrznymi systemami	4
3	Diagramy przypadków użycia	5
3.1	Diagram ogólny	5
3.2	Opis wybranych przypadków użycia	5
4	Diagram komponentów	8
4.1	Diagram komponentów	8
4.2	Opis komponentów	8
5	Diagram klas	9
5.1	Diagram klas	9
5.2	Opis kluczowych struktur danych	9
6	Diagramy interakcji	10
6.1	Diagram interakcji: Osadzenie znaku wodnego z klasyfikacją AI	10
6.2	Diagram interakcji: Zakup / Upgrade planu subskrypcji	10
7	Stack technologiczny	12
8	Działanie systemu	13
8.1	Zrzuty ekranu interfejsu użytkownika	13
9	Podsumowanie	15

1 Wprowadzenie

1.1 Cel projektu

Głównym celem projektu **StegoCloud** jest zaprojektowanie oraz implementacja nowoczesnego, rozproszonego i bezpiecznego systemu do dystrybucji i weryfikacji obrazów cyfrowych za pomocą ukrytych (niewidocznych) znaków wodnych. System pozwala autorom oraz dystrybutorom treści multimedialnych (np. obrazów PNG) na weryfikację ich oryginalności oraz identyfikację ewentualnego źródła przecieku lub nieautoryzowanego użycia, przy jednoczesnym zabezpieczeniu samej treści znaku wodnego przed odczytem przez osoby nieuprawnione.

1.2 Zakres projektu

System został zaprojektowany w nowoczesnej architekturze mikroservisowej i obejmuje:

- **Usługi autoryzacji i rejestracji** zarządzające tożsamościami użytkowników i ról (Zwykły Użytkownik, Administrator).
- **Moduł subskrypcyjny** realizujący logikę biznesową planów taryfowych (FREE, STANDARD, PRO) oraz dynamiczne zarządzanie saldem i rezerwacjami tokenów użytkowników.
- **Serwis znakowania (watermarkingu)** oparty o zaawansowany silnik w języku Python osadzający zaszyfrowane komunikaty w dziedzinie częstotliwościowej (DCT) obrazów PNG.
- **Serwis sztucznej inteligencji (AI)** dokonujący klasyfikacji zawartości obrazów przy użyciu modelu MobileNetV2 w środowisku uruchomieniowym ONNX.
- **Interfejs graficzny użytkownika (GUI)** zrealizowany w technologii Svelte, pozwalający na łatwe korzystanie z funkcji platformy.

1.3 Grupa docelowa (Do kogo projekt jest skierowany)

StegoCloud jest skierowany przede wszystkim do:

- **Twórców cyfrowych i fotografów** chcących trwale zabezpieczyć swoje prace przed kradzieżą i niekontrolowanym rozpowszechnianiem.
- **Agencji reklamowych i marketingowych** dystrybuujących materiały graficzne do klientów zewnętrznych, które muszą pozostać poufne do momentu oficjalnej publikacji.
- **Systemów zarządzania treścią (CMS) i platform e-commerce** w celu automatycznego stemplowania pobieranych obrazów tożsamością kupującego (śledzenie nielegalnej redystrybucji).

2 Opis systemu

2.1 Przeznaczenie

Platforma StegoCloud służy do bezpowrotnego, niewidocznego dla ludzkiego oka stemplowania cyfrowych plików PNG. W przeciwieństwie do widocznych znaków wodnych, które można łatwo wyciąć lub zamazać, steganografia w dziedzinie częstotliwościowej (Discrete Cosine Transform – DCT) osadza informacje bezpośrednio w strukturze pikseli obrazu. Znak wodny chroniony jest kryptograficznie (AES-GCM) oraz nadmiarowo (Reed-Solomon Error Correction Code), co zapobiega fałszowaniu i pozwala na poprawny odczyt nawet przy drobnych zniekształceniach.

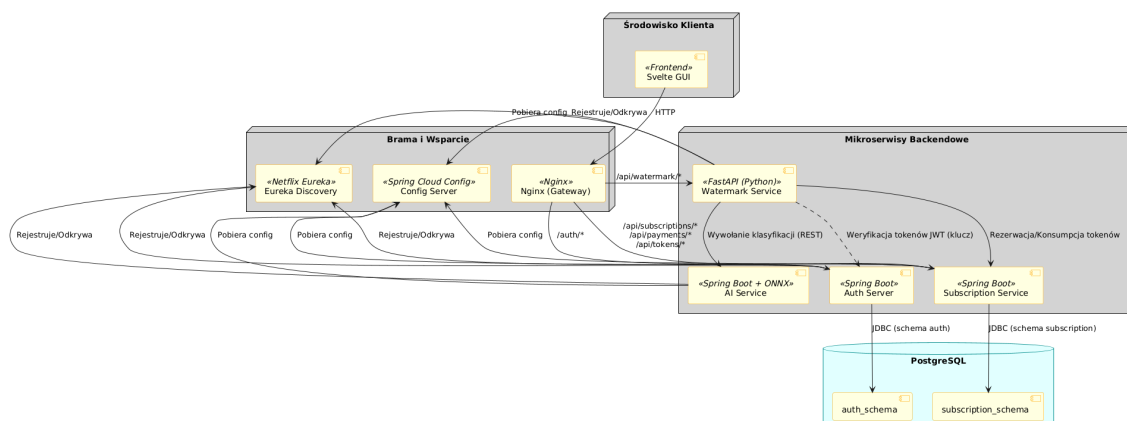
2.2 Główne funkcje

System oferuje następujące operacje taryfikowane odpowiednim kosztem tokenowym:

1. **Capacity Check (0 tokenów)**: Analizuje obraz PNG i informuje o maksymalnej liczbie bajtów, jakie można w nim ukryć w zależności od jego wymiarów (minimalny rozmiar to 1024×1024 pikseli).
2. **Embed (5 lub 8 tokenów)**: Osadza zaszyfrowany i zabezpieczony kodem korekcyjnym tekst w obrazie. Rozmiary poniżej Full HD (1920×1080) zużywają 5 tokenów (tier `EMBED_768`), większe 8 tokenów (tier `EMBED_1024`).
3. **Detect (1 token)**: Sprawdza, czy w przesłanym obrazie znajduje się poprawny znak wodny i określa jego właściciela.
4. **Extract (2 tokeny)**: Wyodrębnia ukryty tekst. Zwykły użytkownik może odczytać tylko swoje znaki wodne, administrator ma uprawnienia do wszystkich.
5. **Visualize (3 tokeny)**: Generuje mapę ciepła (heatmap) pokazującą bezwzględne różnice wartości pikseli przed i po osadzeniu znaku wodnego.
6. **AI Classification (2 tokeny)**: Opcjonalna klasyfikacja zawartości obrazu przy pomocy modelu sieci neuronowej MobileNetV2 (dostępna tylko dla planu `PRO`).

2.3 Architektura systemu

Projekt zrealizowano w architekturze mikroserwisowej z wykorzystaniem stosu technologicznego Spring Cloud oraz kontenerów Docker. Diagram przedstawia strukturę powiązań pomiędzy komponentami systemu.



Rysunek 1: Architektura mikroserwisowa systemu StegoCloud

2.4 Integracje z zewnętrznymi systemami

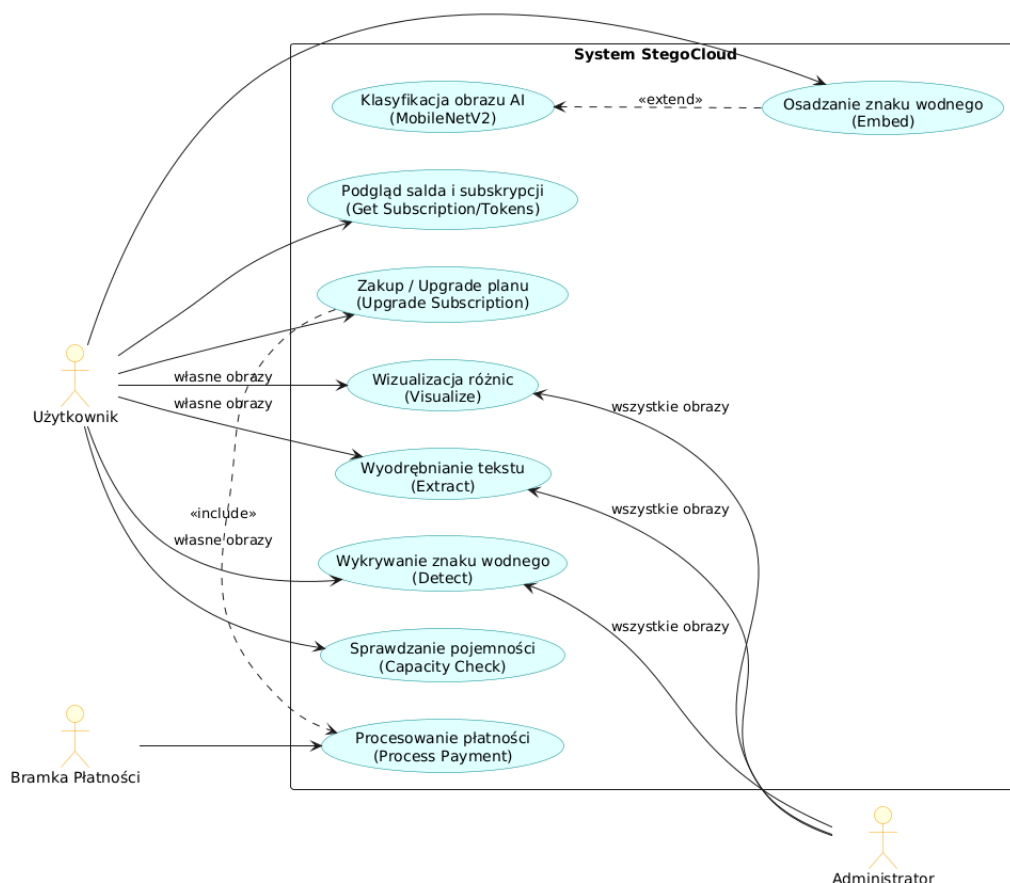
System integruje się z następującymi mechanizmami wspierającymi:

- **Spring Cloud Config Server:** Centralne repozytorium konfiguracyjne przechowujące pliki konfiguracyjne mikroserwisów w dedykowanym repozytorium Git. *Zmiany konfiguracji mogą być dynamicznie propagowane.*
- **Spring Cloud Netflix Eureka:** Serwer rejestracji i odkrywania usług (Service Discovery), umożliwiający dynamiczne kierowanie ruchu i skalowalność mikroserwisów.
- **PostgreSQL:** Relacyjna baza danych współdzielona na poziomie instancji, ale logicznie wydzielona na osobne schematy: `auth_schema` dla serwera autoryzacji oraz `subscription_schema` dla serwisu subskrypcji.
- **ONNX Runtime:** Zintegrowane bibliotecznie w serwisie AI środowisko wykonawcze do uruchamiania wytrenowanego modelu głębokiego uczenia MobileNetV2 bez konieczności instalowania pełnych bibliotek takich jak TensorFlow czy PyTorch.

3 Diagramy przypadków użycia

3.1 Diagram ogólny

Diagram przypadków użycia przedstawia interakcje aktorów (Użytkownik, Administrator oraz zewnętrzna Bramka Płatności) z poszczególnymi funkcjonalnościami systemu StegoCloud.



Rysunek 2: Diagram przypadków użycia systemu StegoCloud

3.2 Opis wybranych przypadków użycia

Poniższe tabele szczegółowo opisują dwa kluczowe przypadki użycia systemu: osadzanie znaku wodnego z klasyfikacją AI oraz zakup/upgrade planu subskrypcji.

Tabela 1: Opis przypadku użycia: Osadzanie znaku wodnego z klasyfikacją AI

Nazwa przypadku	Osadzenie znaku wodnego z klasyfikacją AI
Aktorzy	Użytkownik Zalogowany, watermark-service-py, subscription-service, ai-service
Warunki wstępne	1. Użytkownik jest zalogowany i posiada ważny token JWT. 2. Użytkownik posiada aktywny plan PRO. 3. Saldo tokenów użytkownika wynosi co najmniej 7 tokenów (koszt embed: 5 lub 8 + koszt AI: 2). 4. Przesłany plik jest poprawnym formatem PNG o wymiarach $\geq 1024 \times 1024$ px.
Przebieg główny	1. Użytkownik wybiera plik PNG oraz wpisuje treść znaku wodnego w GUI. 2. GUI wysyła żądanie do bramki Nginx, która przekazuje je do watermark-service-py. 3. watermark-service-py pyta subscription-service o rezerwację tokenów na operację embedowania. 4. subscription-service dokonuje rezerwacji tokenów o statusie PENDING. 5. watermark-service-py pyta subscription-service o rezerwację tokenów na klasyfikację AI. Rezerwacja zostaje zatwierdzona. 6. watermark-service-py wysyła obraz do ai-service. 7. ai-service uruchamia model ONNX, klasyfikuje obraz i zwraca etykiety (np. klasyfikacja: "samochód", pewność: 94%). 8. watermark-service-py przesyła żądanie konsumpcji rezerwacji AI. 9. watermark-service-py szyfruje payload AES-GCM, koduje Reed-Solomon i osadza go w DCT obrazu. 10. Po poprawnym osadzeniu, serwis wysyła żądanie konsumpcji rezerwacji embedowania do subscription-service. 11. Obraz PNG wraz z nagłówkami wyników klasyfikacji AI zostaje przesłany do GUI i udostępniony do pobrania.
Przebieg alternatywny	4a/5a. Brak tokenów lub niepoprawny plan: subscription-service odmawia rezerwacji. Proces zostaje przerwany z błędem 403/409, użytkownik widzi komunikat w GUI. 9a. Błąd algorytmu steganograficznego: watermark-service-py zwalnia rezerwacje tokenów w subscription-service (status RELEASED). Saldo użytkownika nie ulega zmianie.

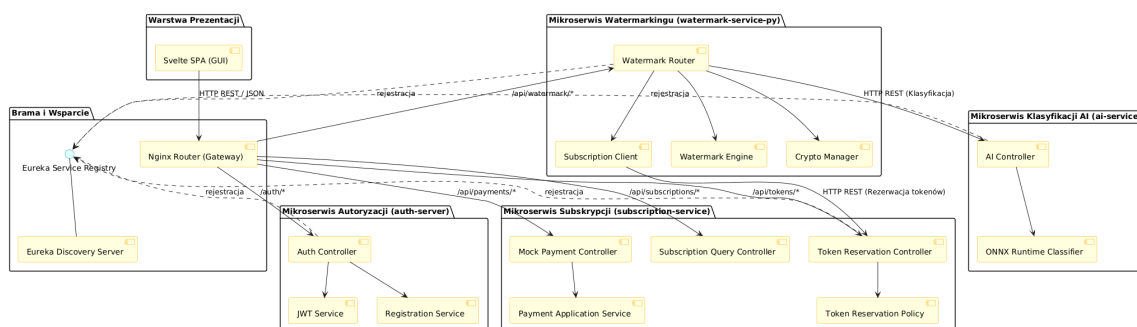
Tabela 2: Opis przypadku użycia: Zakup / Upgrade planu subskrypcji

Nazwa przypadku	Zakup / Upgrade planu subskrypcji
Aktorzy	Użytkownik Zalogowany, <code>subscription-service</code> , Bramka Płatności (Mock)
Warunki wstępne	1. Użytkownik jest zalogowany. 2. Użytkownik posiada aktualnie niższy plan niż docelowy (dozwolone przejścia: <code>FREE</code> → <code>STANDARD</code>, <code>FREE</code> → <code>PRO</code>, <code>STANDARD</code> → <code>PRO</code>).
Przebieg główny	1. Użytkownik w panelu subskrypcji wybiera przycisk "Kup" przy wybranym nowym planie. 2. GUI wysyła żądanie utworzenia sesji płatniczej do <code>subscription-service</code>. 3. <code>subscription-service</code> tworzy sesję płatniczą o statusie <code>PENDING</code> i zwraca jej identyfikator oraz adres bramki płatniczej. 4. GUI przekierowuje użytkownika na stronę bramki płatności (Mock Payment Page). 5. Użytkownik klika przycisk "Zatwierdź płatność". 6. Bramka płatności wysyła żądanie sukcesu płatności do <code>subscription-service</code> przekazując <code>sessionId</code>. 7. <code>subscription-service</code> weryfikuje sesję i jej właściciela, a następnie zmienia status sesji na <code>SUCCESS</code>. 8. Serwis modyfikuje subskrypcję użytkownika w bazie (nowy plan, nowa data ważności na +1 miesiąc). 9. Saldo tokenów użytkownika zostaje zasilone pełną pulą nowego planu. 10. Użytkownik zostaje przekierowany z powrotem do aplikacji, gdzie GUI prezentuje zaktualizowany stan konta.
Przebieg alternatywny	1a. Downgrade lub zakup tego samego planu: Backend blokuje operację (błąd 400), informując o niedozwolonym przejściu. 5a. Anulowanie płatności: Użytkownik klika "Anuluj". Bramka wysyła status anulowania. Status sesji w bazie zmienia się na <code>CANCELLED</code>. Subskrypcja użytkownika pozostaje bez zmian.

4 Diagram komponentów

4.1 Diagram komponentów

Diagram komponentów przedstawia dekompozycję systemu na poszczególne moduły aplikacyjne, interfejsy komunikacyjne oraz zależności między nimi.



Rysunek 3: Diagram komponentów systemu StegoCloud

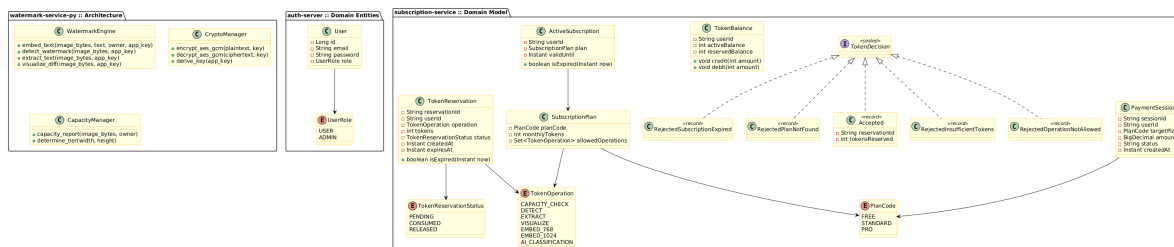
4.2 Opis komponentów

- **Svelte SPA (GUI):** Aplikacja kliencka uruchamiana w przeglądarce użytkownika. Odpowiada za prezentowanie formularzy, obsługę plików PNG, podgląd heatmap i zarządzanie profilami subskrypcji.
- **Nginx Router (Gateway):** Odpowiada za kierowanie ruchu zewnętrznego (routing) do odpowiednich mikroserwisów na podstawie prefiksów ścieżek URL, pełniąc funkcję API Gateway.
- **Auth Controller & JWT Service:** Części składowe `auth-server`. Odpowiadają za weryfikację haseł (BCrypt), generowanie tokenów JWT oraz rejestrację nowych użytkowników.
- **Token Reservation & Subscription Controller:** Komponenty `subscription-service`. Odpowiadają odpowiednio za przyjmowanie żądań rezerwacji tokenów oraz zarządzanie sesjami płatniczymi i planami.
- **Watermark Router & Engine:** Komponenty usługi `watermark-service-py`. Router wystawia punkty końcowe (REST API), a Engine wykonuje algorytm częstotliwościowy przy użyciu biblioteki `invisible-watermark`.
- **Crypto Manager:** Komponent pomocniczy usługi watermarkingu. Szyfruje tekst znaku wodnego z użyciem AES-GCM przed osadzeniem, używając klucza wyprowadzonego z sekretu aplikacji.
- **AI Controller & ONNX Runtime Classifier:** Części składowe `ai-service`. Pobierają obraz i przekazują go do silnika ONNX Runtime uruchamiającego sieć MobileNetV2, która wykonuje klasyfikację.

5 Diagram klas

5.1 Diagram klas

Diagram klas prezentuje kluczowe encje bazodanowe, rekordy oraz interfejsy wchodzące w skład warstwy domenowej systemów `subscription-service`, `auth-server` oraz strukturę modułu w `watermark-service-py`.



Rysunek 4: Diagram klas domenowych systemu StegoCloud

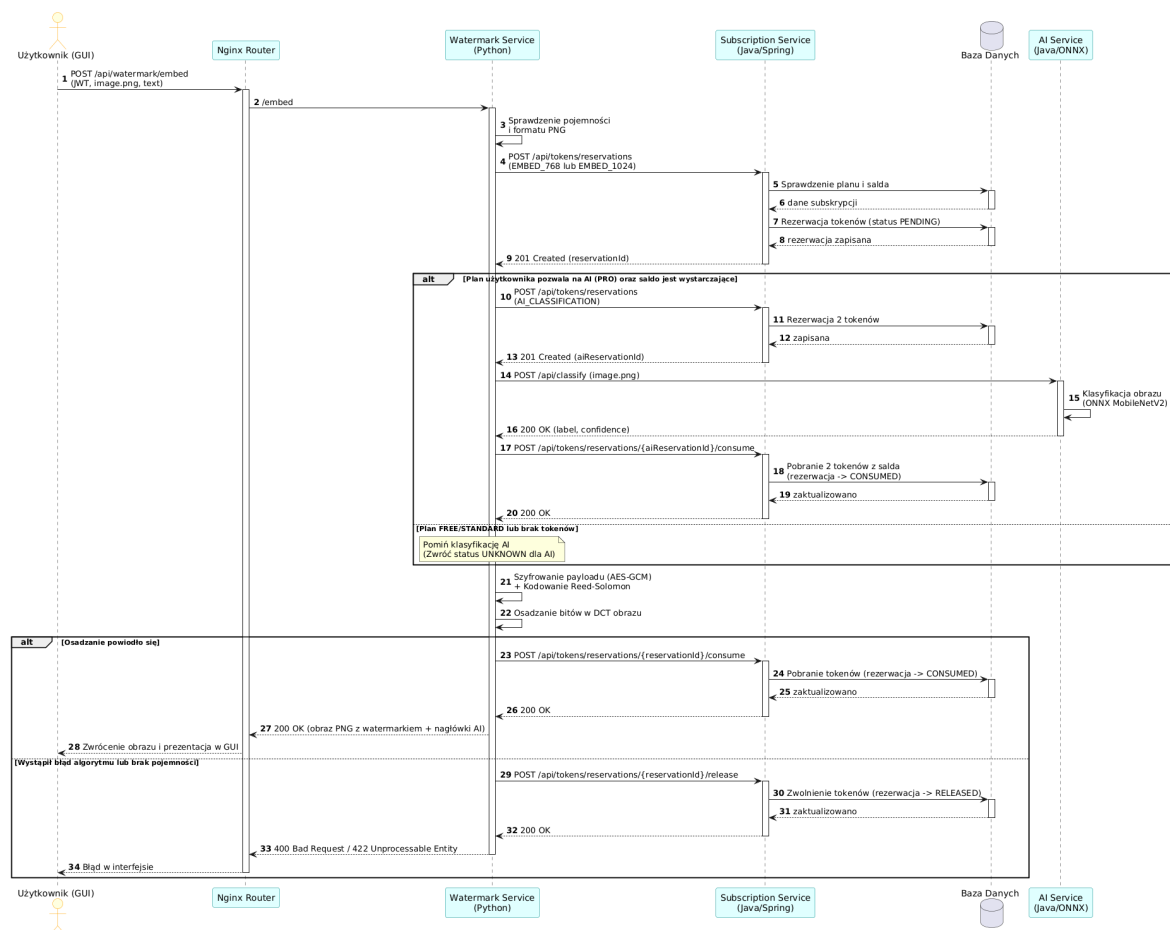
5.2 Opis kluczowych struktur danych

- **User & UserRole:** Reprezentują użytkownika w systemie autoryzacji wraz z rolą przypisaną na potrzeby kontroli dostępu na poziomie endpointów (RBAC).
- **SubscriptionPlan:** Klasa definiująca właściwości planu taryfowego – liczbę darmowych tokenów na miesiąc oraz zbiór dozwolonych operacji (`allowedOperations`).
- **ActiveSubscription:** Encja reprezentująca powiązanie użytkownika z konkretnym planem subskrypcyjnym wraz z okresem jego ważności. Posiada metodę `isExpired()` określającą ważność subskrypcji.
- **TokenBalance:** Encja reprezentująca bilans tokenów zalogowanego użytkownika. Zawiera informację o tokenach dostępnych (`activeBalance`) oraz zablokowanych na poczet trwających operacji (`reservedBalance`).
- **TokenReservation:** Reprezentuje rezerwację tokenów dokonaną przez system podczas rozpoczęcia operacji steganograficznej. Posiada unikalne ID, typ operacji, koszt tokenowy oraz status (`PENDING`, `CONSUMED`, `RELEASED`).
- **TokenDecision:** Zamknięty interfejs (`sealed interface`), który przy użyciu mechanizmów Javy 21 wymusza kompletną obsługę wszystkich możliwych decyzji biznesowych dotyczących przydziału tokenów (akceptacja lub konkretny powód odmowy).

6 Diagramy interakcji

6.1 Diagram interakcji: Osadzenie znaku wodnego z klasyfikacją AI

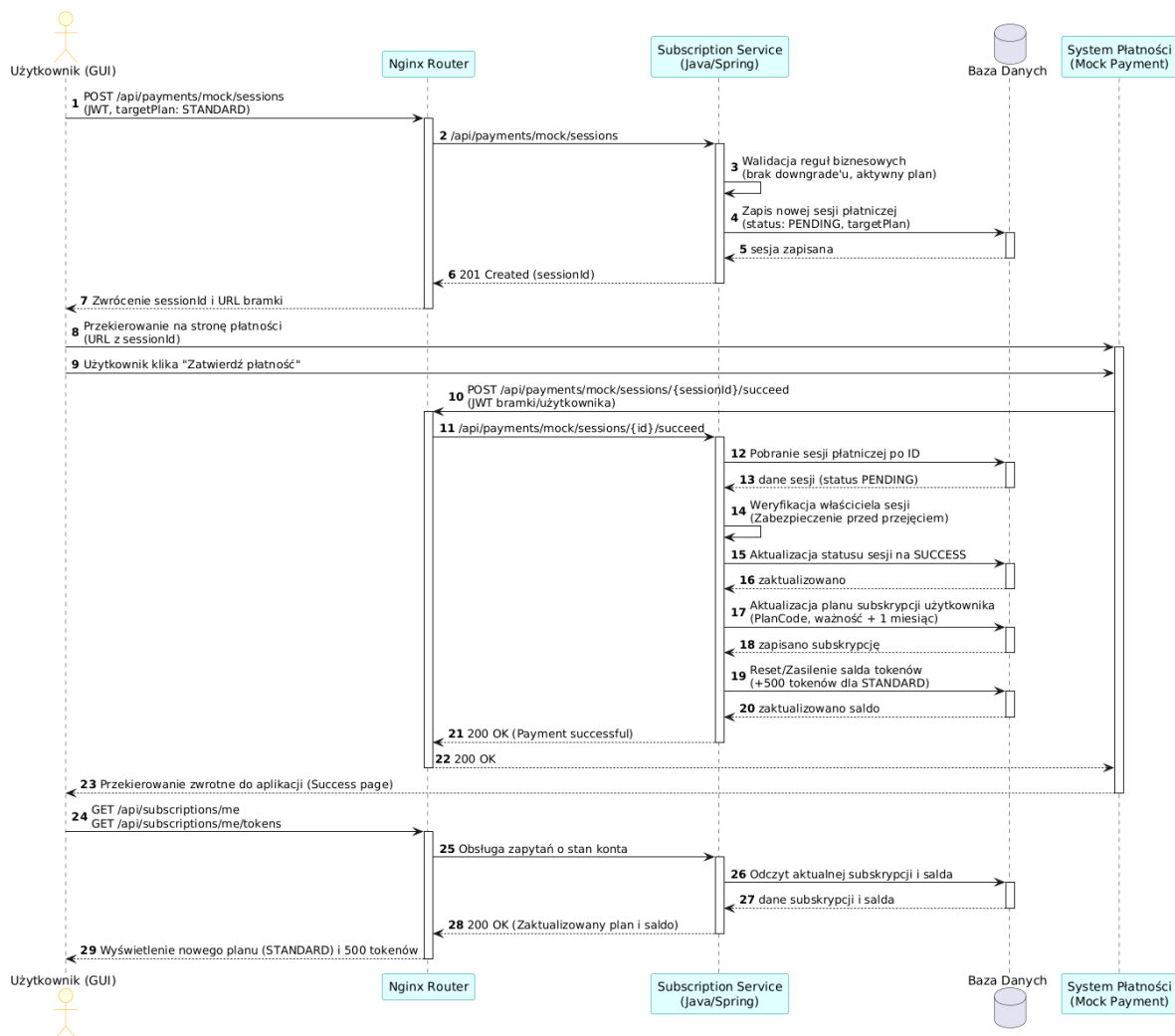
Diagram przedstawia sekwencję komunikatów i wywołań API podczas osadzania tekstu w obrazie z opcjonalną weryfikacją zawartości przez sztuczną inteligencję.



Rysunek 5: Diagram sekwencyjny: Proces osadzania znaku wodnego z klasyfikacją AI

6.2 Diagram interakcji: Zakup / Upgrade planu subskrypcji

Diagram przedstawia przebieg rejestracji sesji płatniczej, jej opłacenia oraz ostatecznej aktualizacji danych abonamentowych w bazie danych.



Rysunek 6: Diagram sekwencyjny: Proces zakupu / upgrade'u planu subskrypcji

7 Stack technologiczny

Wybrane komponenty systemu zostały zaimplementowane przy użyciu technologii dobranych pod kątem ich zalet wydajnościowych oraz spełnienia celów dydaktycznych i biznesowych projektu.

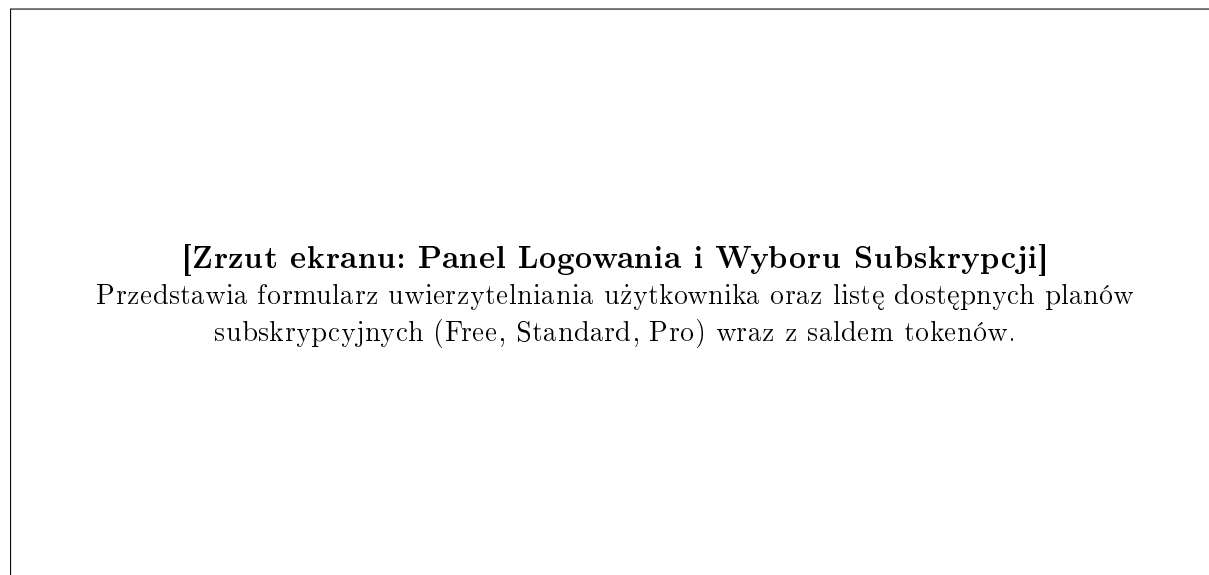
Tabela 3: Wykorzystane technologie i uzasadnienie wyboru

Komponent	Technologia	Uzasadnienie wyboru
Backend Core	Java 21 / Spring Boot 3	Zapewnia stabilne środowisko, wysokie bezpieczeństwo typów oraz wsparcie dla najnowszych funkcjonalności JDK (rekordy, dopasowywanie wzorców dla switcha, sealed classes), które upraszczają kod domeny subskrypcji.
Steganografia	Python 3.11 / FastAPI	Znakowanie obrazów wymaga niskopoziomowych operacji matematycznych (DCT) oraz wsparcia dla zaawansowanych bibliotek naukowych (NumPy, invisible-watermark). FastAPI zapewnia asynchroniczność oraz generuje automatyczną dokumentację OpenAPI.
AI/Klasyfikacja	ONNX Runtime (Java)	Umożliwia ładowanie i szybkie uruchamianie modeli sieci neuronowych (MobileNetV2 wyeksportowanego z PyTorch) bezpośrednio w procesie Javy z optymalizacją sprzętową, bez potrzeby stawiania ciężkiego środowiska PyTorch.
Baza Danych	PostgreSQL 16	Stabilna, relacyjna baza danych obsługująca transakcje ACID niezbędne w logice rozliczania tokenów i płatności. Zapewnia izolację za pomocą osobnych schematów bazodanowych.
Frontend GUI	Svelte 4 / nginx	Svelte generuje niezwykle lekki kod kliencki (brak narzutu Virtual DOM), co przekłada się na szybkie ładowanie interfejsu. Nginx służy jako serwer statyczny i Gateway proxy.
Narzędzia	Docker Compose / SonarQube	Konteneryzacja ułatwia lokalne uruchamianie całego rozproszonego środowiska. SonarQube umożliwia ciągłą analizę statyczną kodu w celu utrzymania długu technicznego na niskim poziomie.

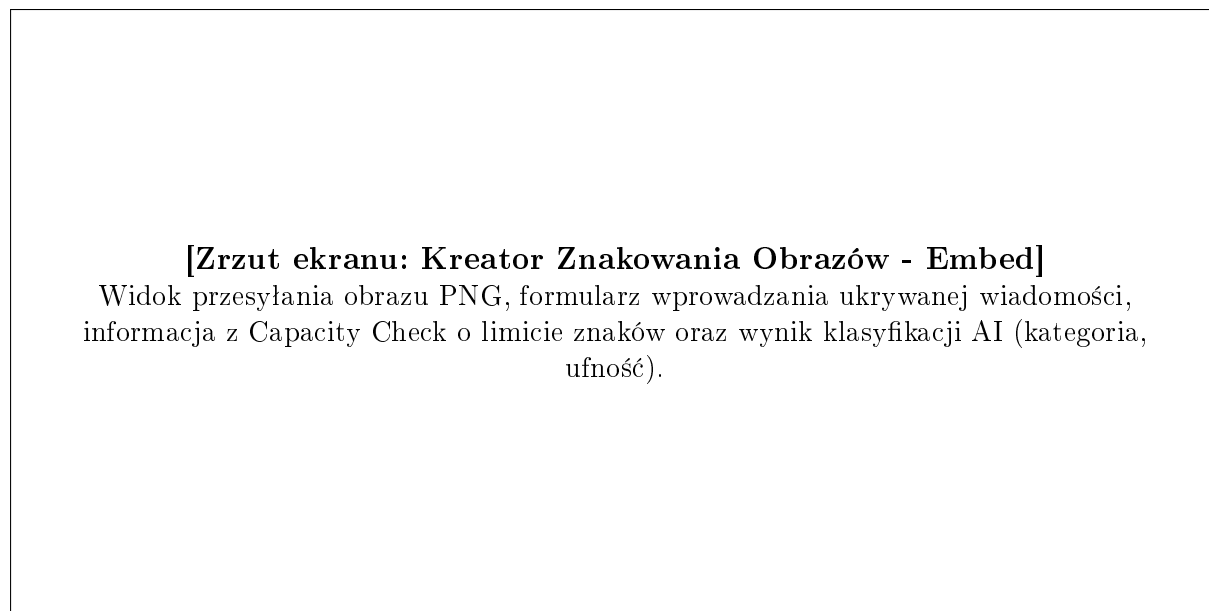
8 Działanie systemu

W tej sekcji przedstawiono kluczowe zrzuty ekranu prezentujące działanie działającego systemu StegoCloud.

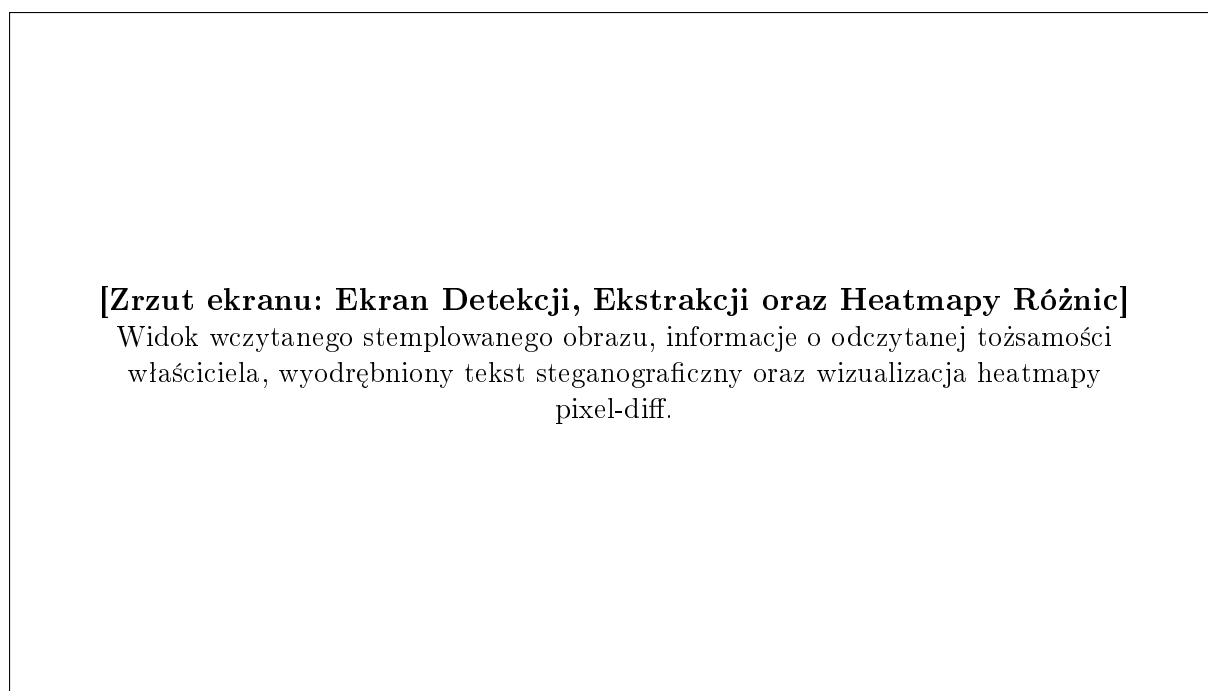
8.1 Zrzuty ekranu interfejsu użytkownika



Rysunek 7: Placeholder: Ekran wyboru planu subskrypcji i logowania



Rysunek 8: Placeholder: Panel osadzania znaku wodnego z klasyfikacją AI



Rysunek 9: Placeholder: Ekran detekcji znaku wodnego i wizualizacji heatmapy różnic

9 Podsumowanie

Projekt **StegoCloud** z powodzeniem integruje zaawansowane mechanizmy platformy Java 21, architekturę mikroserwisową Spring Cloud oraz dedykowany silnik przetwarzania obrazów w języku Python.

Dzięki zastosowaniu wzorca rezerwacji tokenów system gwarantuje odporność na błędy sieciowe i transakcyjne – użytkownicy są obciążani tokenami wyłącznie za operacje, które zostały pomyślnie zakończone. Zastosowanie szyfrowania kopertowego AES-GCM oraz Reed-Solomon ECC w warstwie steganografii gwarantuje wysoki poziom poufności i odporności na zakłócenia. System spełnia wymagania stawiane nowoczesnym aplikacjom korporacyjnym, a jego modularna budowa pozwala na łatwe dodawanie kolejnych algorytmów znakowania oraz modeli klasyfikacji AI w przyszłości.